Smallest enclosing balls by an active-set method on the dual

Thomas Schmelzer

September 4, 2026

Abstract

We record the dual quadratic program of the smallest enclosing ball problem, the active-set method that solves it, and the invariance properties an implementation must preserve. The method terminates at an exact vertex of the dual feasible set rather than at an interior-point tolerance, costs one $k \times k$ dense solve per iteration with $k \le d$, and returns the dual weights, which are the conic dual of the second-order cone formulation after one rescaling and hence a certificate the caller can check independently.

1 The problem and its dual

For $S = \{p_1, \dots, p_n\} \subset \mathbb{R}^d$ the smallest enclosing ball is the solution of

$$\min_{x,r} \; r \quad \text{s.t.} \quad \|p_i - x\| \le r, \; i = 1, \dots, n, \tag{1}$$

a second-order cone program in $(r, x) \in \mathbb{R}^{1+d}$ with n blocks $(r, p_i - x) \in \mathcal{Q}^{d+1}$. Existence follows from coercivity of $f(x) = \max_i \|p_i - x\|$ and uniqueness from strict convexity of f^2 along any segment; we write $(R, x^\star)$ for the optimum.

Let $\Delta_n = \{u \in \mathbb{R}^n : u \ge 0, \; \mathbf{1}^\mathsf{T} u = 1\}$, and for $u \in \Delta_n$ put $x(u) = \sum_i u_i p_i$ and

$$\varphi(u) = \sum_i u_i \|p_i\|^2 - \|x(u)\|^2, \tag{2}$$

a concave quadratic. Everything below rests on one identity, valid for all $y \in \mathbb{R}^d$ and $u \in \Delta_n$:

$$\sum_i u_i \|p_i - y\|^2 \;=\; \sum_i u_i \|p_i\|^2 - 2\, x(u)^\mathsf{T} y + \|y\|^2 \;=\; \varphi(u) + \|x(u) - y\|^2. \tag{3}$$

Both steps are mechanical. The first expands the square and collects terms, using $\sum_i u_i p_i = x(u)$ on the cross term and $\mathbf{1}^\mathsf{T} u = 1$ on $\|y\|^2$; the second adds and subtracts $\|x(u)\|^2$, which by (2) leaves $\varphi(u)$ beside a completed square. What makes it an equality rather than a bound is that no term has been discarded: the weights enter only through $x(u)$ and through $\sum_i u_i \|p_i\|^2$, and φ is defined as exactly the difference of those two. So this is the parallel-axis identity of mechanics, or the bias–variance decomposition: measuring from anywhere other than the weighted mean costs precisely $\|x(u) - y\|^2$, whatever the weights, and $\varphi(u)$ is what is left when y is the mean.

Proposition 1. *$R^2 = \max_{u \in \Delta_n} \varphi(u)$, and $x^\star = x(w)$ for any maximiser w.*

Proof. If $B(y, r) \supseteq S$ then the left side of (3) is at most r^2, so $\varphi(u) \le r^2$ for every u; taking the optimum gives $\max \varphi \le R^2$. Conversely let $T = \{p_i : \|p_i - x^\star\| = R\}$. If $x^\star \notin \operatorname{conv} T$, separate: there is v with $v^\mathsf{T}(p - x^\star) > 0$ on T, and $\|p - (x^\star + tv)\|^2 = R^2 - 2tv^\mathsf{T}(p - x^\star) + t^2\|v\|^2 < R^2$ for small $t > 0$, while the finitely many points off T stay strictly inside — so the radius decreases, a contradiction. Hence $x^\star = \sum_i w_i p_i$ for some $w \in \Delta_n$ supported on T, and $y = x^\star$ in (3) gives $\varphi(w) = \sum_i w_i \|p_i - x^\star\|^2 = R^2$. The same substitution forces $\|x(w) - x^\star\|^2 \le 0$ for any maximiser. □

Since $\partial\varphi / \partial u_i = \|p_i - x(u)\|^2 - \|x(u)\|^2$, the first-order conditions for $\max_{\Delta_n} \varphi$ read, after cancelling the common term,

$$\|p_i - x\| = R \;\; (w_i > 0), \qquad \|p_i - x\| \le R \;\; (w_i = 0), \tag{4}$$

the geometric certificate: the points carrying weight lie on the boundary sphere and the rest are enclosed. By Carathéodory the support may be taken of size at most $d + 1$, so (1) is determined by at most $d + 1$ of the n points and the problem is combinatorial in the choice of that subset.

2 The method

Maintain a working set F and weights $u > 0$ on it summing to one.

Subproblem. Maximising φ over weights supported on F with no sign restriction is, by (4) without the inequalities, the requirement that x be equidistant from $\{q_j\}_{j\in F}$ and lie in their affine hull. With D the $(m-1)\times d$ matrix of edges $e_j = q_j - q_0$ and $x = q_0 + D^{\mathsf T} y$,

$$(DD^{\mathsf T})\, y = \tfrac{1}{2}\big(\|e_1\|^2, \ldots, \|e_{m-1}\|^2\big)^{\mathsf T}, \qquad w = \Big(1 - \textstyle\sum_j y_j,\ y\Big), \tag{5}$$

a dense system of order $m-1 \le d$, non-singular exactly when F is affinely independent. This is the circumcentre of the face, expressed in barycentric coordinates.

Dropping. If $\min_j w_j < 0$ the circumcentre lies outside $\operatorname{conv}\{q_j\}$ and the subproblem solution is dual-infeasible. Move along $s = w - u$ by the ratio test

$$\alpha = \min\{-u_i/s_i :\ s_i < 0\}, \tag{6}$$

which drives at least one weight to zero; remove it from F.

Degeneracy. If F is affinely dependent, $DD^{\mathsf T}$ is singular. The directions s with $\mathbf{1}^{\mathsf T}s = 0$ and $\sum_i s_i q_i = 0$ — equivalently $D^{\mathsf T}s_{1:} = 0$ after eliminating s_0 — leave $x(u)$ fixed, so $\|x(u)\|^2$ is frozen and $\varphi(u + ts) = \varphi(u) + t\sum_i s_i\|p_i\|^2$ is linear: the subproblem is unbounded. Project the gradient onto that null space and apply (6), shrinking F by one.

Adding. Otherwise accept $u \leftarrow w$, set $x = x(u)$ and $R = \max_{i\in F}\|p_i - x\|$, and let $k = \arg\max_i \|p_i - x\|$. If $\|p_k - x\| \le R$ then (4) holds and (R, x, u) is optimal; otherwise append k to F with weight zero. By (7) below this is also the steepest-ascent choice among the fixed variables.

> **Input** $p_1, \ldots, p_n$. Recentre on $\bar p$. Set $F = \{k\}$, k farthest from p_1; $u_k = 1$.
> **Repeat:** if F affinely dependent, take the null-space descent step and drop; else solve (5). If $\min w < 0$, take the ratio step (6) and drop. Else set $u = w$, $x = x(u)$, $R = \max_{i\in F}\|p_i - x\|$; if no point exceeds R, return $(R,\ x + \bar p,\ u)$; else drop zero weights and append $\arg\max_i \|p_i - x\|$ with weight 0.

Each iteration either strictly increases φ or strictly shrinks F, and there are finitely many working sets, so the method is finite; as with any active-set method a degenerate zero-length step requires an anti-cycling rule in exact arithmetic — for this problem, a Bland-type order on the points, which Fischer, Gärtner and Kutz [2] prove terminating and then reject as too slow. In practice an iteration cap serves as a safety net. Note that

$$\partial\varphi/\partial u_k - \partial\varphi/\partial u_i = \|p_k - x\|^2 - \|p_i - x\|^2, \tag{7}$$

so the farthest violator is the fixed variable with the largest reduced gradient.

3 Invariance

Two properties are load-bearing and easily lost. Both are consequences of refusing any tolerance tied to coordinates of magnitude one.

Origin invariance. Recentre on $\bar p$ before iterating, and form $\|p - x\|^2$ as $\sum_k (p_k - x_k)^2$ rather than $\|p\|^2 - 2p^{\mathsf T}x + \|x\|^2$. The expanded form cancels quantities of size $\|x\|^2$ to produce an answer of size R^2, losing every digit by which $\|x\|$ exceeds R; for a cloud of unit extent at distance 10^6 the rounding floor of a squared distance is then of the order of the radius itself.

Scale invariance. Test affine dependence on the rank of D, not on $[\mathbf{1}; Q^{\mathsf T}]$: the row of ones has unit entries, dominates the singular values, and would declare any cloud of extent below ε degenerate. Size the feasibility slack of the stopping test as a relative tolerance plus an absolute floor proportional to $\varepsilon \max_i\|p_i\|^2$, and normalise the null-space direction before comparing anything derived from it — the gradient carries units of length^2, the weight tolerance does not.

Exponent range. Tolerances are not the whole of it. A double reaches to 10^{308}, so a *squared* double reaches only to 10^{154}, and this method squares everything: the Gram matrix, the distances, the noise floor. A cloud of extent 10^{-160} — ordinary coordinates — therefore has a Gram matrix in the subnormals, and the solve (5) overflows; at 10^{+160} the squares saturate and the method reports $R = 0$ without raising. Rescaling the centred cloud to unit extent removes both, and the factor must be a power of two: any other rounds, and the exactness Table 1 reports is lost with it.

4 Cost, and the dual as a certificate

Per iteration the method performs one $O(nd)$ sweep of squared distances and one dense solve of order at most d; nothing of size n is ever factorised. This is the structural difference from an interior-point method applied to (1), which factors an $n(d+1)$-row system at every step. The method also identifies the support combinatorially and solves that face exactly, rather than approaching it from the interior and leaving the caller to decide which slacks of size 10^{-9} were meant to be zero.

The weights are the conic dual of (1) after one rescaling: the block for point i is

$$z_i = u_i\Big(1,\ -\frac{p_i - x}{R}\Big) \in \mathcal{Q}^{d+1}, \tag{8}$$

whose tail has norm $u_i\|p_i - x\|/R$, equal to its head exactly on the support, and which pairs with the slack $s_i = (R,\, p_i - x)$ as $s_i^{\mathsf{T}} z_i = (u_i/R)(R^2 - \|p_i - x\|^2) = 0$ — complementarity, each factor vanishing where the other does not. So a solver built on this method returns a standard-form primal–dual pair, and a caller can verify optimality from (4) at the cost of one pass over the data, without re-solving.

Remark (Substitutability). This is what makes the method usable behind a conic solver's interface rather than only as a special-purpose routine. The accompanying implementation dispatches on the cone list: zero and non-negative blocks go to a dense dual active-set QP method [5], and equally sized second-order blocks matching the shape of (1) go to the method above. A program carrying the right cones but a different matrix is refused rather than answered, which requires reconstructing the point cloud from (A, b) and rebuilding the program for comparison. An indefinite objective is likewise refused rather than regularised into definiteness, since $P + \epsilon I$ answers a different question.

5 A numerical comparison

Table 1 compares the method with Clarabel 0.11.1 applied to (1), with Welzl's algorithm [9], and with the pivoting method of Fischer, Gärtner and Kutz [2] discussed in Section 8, on Gaussian clouds. Two quantities are reported, because either alone would mislead.

The first is the relative enclosure error $\max_i \|p_i - x\|/R - 1$, where a positive value means the returned ball does not in fact contain the cloud. It is a deterministic property of the returned pair, free of timing noise. Here the three combinatorial methods are indistinguishable — all come in at zero or a single ulp, against Clarabel's interior-point residuals three to five orders larger — because each returns a radius that is by construction the maximum distance over its own support. The exactness is thus a property of terminating at a basis, not of this method in particular.

One ulp of that is worth naming, since it is the only place the three separate. The method of Section 2 solves afresh for the circumcentre of its final support and lands on zero; the pivoting method reports the running sum of the walks that carried its centre there, and so shows a single positive ulp on four of the seven rows. Both identify the same support set. The difference is in the arithmetic that produces the centre, not in the combinatorics that select it, and at $2.2 \cdot 10^{-16}$ it is of no practical consequence — it is recorded only because a column of exact zeros beside a column of ulps invites the wrong inference.

The second is wall-clock time, and it must be read as a comparison of *implementations*. The method of Section 2 and the pivoting method each spend their time in one vectorised sweep per iteration, whereas Welzl's recursion evaluates a containment predicate per step in scalar Python; a good deal of the gap in the t columns is CPython against NumPy rather than one algorithm against another. A step count is therefore given alongside each combinatorial method that has an informative one: the circumspheres Welzl computes, and the pivot steps the walking method takes. Both are properties of the algorithm and the input in any language — but they count different things, a solve over at most $d+1$ points against a projection plus a sweep over all n, and so track each method against itself rather than against the other.

Table 2 isolates the dependence on d at fixed n. Welzl's median basis count rises from 191 to 275 574 between $d = 2$ and $d = 11$, a factor between 1.5 and 2.7 per dimension across the upper half of that range, while both simplex-like methods stay under a millisecond throughout: their work per iteration is a solve or a projection of order at most d, and d is small.

The two step counts are what the sweep is really for, and they part company completely. Welzl's grows by three orders of magnitude over these ten dimensions; the pivot count grows from 5 to 21, by a factor of four. That is the separation the pivoting method was built for, and it is visible here at $d \le 11$ even though the regime it was built for begins far higher. The worst-seed lines matter as much as the medians. At $d = 11$ one of fifteen clouds cost 1.6 million basis computations, six times the median, where the worst pivot count is 52 against a median of 21 — a factor of 2.5. Both distributions have a tail that a single measurement will not find, which is the practical form the expected-time analysis takes; the tails are simply not the same size.

		enclosure error				time (s)				steps	
n	d	act. set	pivoting	Clarabel	Welzl	act. set	pivoting	Clarabel	Welzl	pivots	bases
10^3	2	0	$-1.1 \cdot 10^{-16}$	$5.3 \cdot 10^{-12}$	$2.2 \cdot 10^{-16}$	0.000	0.000	0.008	0.010	3	265
10^3	5	$-2.2 \cdot 10^{-16}$	$2.2 \cdot 10^{-16}$	$-7.2 \cdot 10^{-12}$	$2.2 \cdot 10^{-16}$	0.000	0.001	0.032	0.018	21	592
10^4	3	0	0	$3.8 \cdot 10^{-12}$	0	0.001	0.001	0.175	0.452	10	2 406
10^4	20	0	$2.2 \cdot 10^{-16}$	$1.0 \cdot 10^{-12}$	—	0.006	0.015	0.983	—	90	—
10^5	2	0	$2.2 \cdot 10^{-16}$	$-8.0 \cdot 10^{-13}$	0	0.003	0.007	1.524	0.953	5	813
10^5	3	0	0	$1.3 \cdot 10^{-10}$	0	0.007	0.010	2.016	4.031	10	5 998
10^5	10	0	$2.2 \cdot 10^{-16}$	$-1.3 \cdot 10^{-12}$	—	0.020	0.051	7.168	—	43	—

Table 1: Standard normal clouds, one per row; median of three runs. Apple M4 Pro, CPython 3.12.13, NumPy 2.5.1, Clarabel 0.11.1 at default tolerances. Welzl was not attempted on the rows marked —; Table 2 measures that limit. The pivoting method runs the heuristic pivot rule of [2], with the QR of the edge matrix rebuilt each pivot rather than updated.

d	2	3	4	5	6	7	8	9	10	11
Welzl bases, median $/10^3$	0.19	1.05	2.50	6.4	12.0	21.4	58.6	115	189	276
Welzl bases, worst $/10^3$	0.55	1.70	4.92	12.9	35.9	49.3	194	577	864	1602
pivots, median	5	6	8	10	15	11	18	18	18	21
pivots, worst	6	15	11	18	25	23	33	34	36	52
Welzl time (s)	0.007	0.026	0.053	0.143	0.236	0.491	1.181	2.675	3.796	5.692
pivoting time (ms)	0.2	0.3	0.4	0.5	1.0	0.8	1.3	0.9	0.9	1.1
active-set time (ms)	0.2	0.3	0.3	0.4	0.5	0.5	1.0	0.6	0.6	0.6

Table 2: Growth in d at $n = 10^3$: medians over 15 seeds, worst seed on the second line of each pair. Medians are necessary — the basis count is high-variance, and a mean over five seeds still puts $d = 11$ below $d = 10$.

None of the three comparisons is fair to its subject, and the third is the least fair. Clarabel is built for large sparse programs of arbitrary shape, and Welzl's recursion is being run in the least favourable language for one. The pivoting method is handicapped more specifically: the implementation measured here reproduces the algorithm but not the data structure, rebuilding the QR of the edge matrix at every pivot where [2] updates it by Givens rotations in $O(d^2)$. Omitting it is deliberate — a Givens sweep in interpreted Python would run slower than one call into LAPACK, so implementing it would misstate their claim rather than test it — but it means the t columns show two NumPy programs of similar shape, one doing avoidable work per pivot at values of d too small for it to matter. At $d = 20$ the pivoting method takes 2.5 times the active-set time; nothing in that ratio should be carried to the $d \sim 10^3$ regime their paper reports, where the discarded data structure is the point. That the specialised method wins on its own problem, in its own regime, against implementations chosen for clarity is the weakest possible claim, and the only one made here.

6 Relation to the minimum-norm point problem

If all points have equal norm ρ — after the standard lift of a kernel problem onto a sphere, say — then (2) reduces to $\varphi(u) = \rho^2 - \|x(u)\|^2$, and maximising it over Δ_n is minimising $\|x\|$ over $\operatorname{conv} S$: the minimum-norm point problem of Wolfe [10] and of the Frank–Wolfe family [8]. The working set is then Wolfe's corral, the subproblem (5) his affine minimiser, the drop step his major cycle. Restoring the linear term $\sum_i u_i \|p_i\|^2$ gives the general case, and is what makes the subproblem a circumcentre rather than a projection.

7 Relation to Welzl's algorithm

Welzl's randomised incremental method [9] computes the same object and, as Table 1 confirms, terminates at an exact basis just as the method above does. The two differ in how that basis is searched.

Welzl's search is combinatorial: it recurses over subsets, drawing a point p at random, solving without it, and re-solving with p forced onto the boundary when p falls outside the ball just computed. The permutation drives the recursion, the only arithmetic is the primitive that circumscribes at most $d + 1$ points, and the analysis is that of LP-type problems — expected $O(n)$ time for fixed d, with a constant known to grow superexponentially in d. The method of Section 2 searches instead by ascent on φ: the weights choose the pivot, a negative barycentric coordinate calling for a drop and the farthest violator for an addition. The search is monotone, local, deterministic, and admits add *and* drop steps; what it does not admit is an expected-time

bound, and Section 2 claims none.

Table 2 is the practical shape of that difference: at fixed n the recursion's cost climbs by a factor of two or so per dimension and carries a long upper tail, while a solve of order d does not notice d in this range. The other difference is the certificate. Welzl's recursion yields the ball and its basis; the method above yields the barycentric weights as well, which are what make (4) checkable in one pass and what become (8) when a conic interface asks for a dual. The weights can be recovered from a basis by solving (5), but they are not a by-product of the search there, whereas here they *are* the search — which is also why a support set warm starts a perturbed cloud and a fresh random permutation cannot.

Gärtner's robust implementation [3] addresses the fragility of the recursion's predicates; Section 3 answers the same question here.

8 Relation to the pivoting method of Fischer, Gärtner and Kutz

The other exact method in wide use [2] shares this one's support set and its drop criterion, and differs from it in the direction of approach.

Their loop invariant is that the current ball contains the cloud: $S \subseteq B(c, T)$, with T on its boundary. Feasibility is never surrendered and the radius falls monotonically to R from above, terminating when c reaches $\operatorname{conv} T$. Here it is dual feasibility that is never surrendered — u stays on Δ_n and φ climbs — so by Proposition 1 the radius is a *lower* bound throughout and the ball on hand fails to contain the cloud until the final iteration. Theirs is a primal feasible-point method, this a dual one in the sense of [5]: every iterate solves some face exactly and none is feasible for the whole problem until the last.

The pivots differ accordingly. They never take a full step: the centre walks along $[c, \operatorname{cc}(T)]$ and is truncated by whichever point first touches the shrinking sphere, which is also the point that enters T. Section 2 takes the whole step to $\operatorname{cc}(F)$ and only then consults the data, admitting the farthest violator. The criterion for leaving is the same in both — a negative barycentric coordinate — but the mechanics are not: a ratio test in weight space here, immediate removal and a resumed walk there.

Degeneracy is the sharpest split. Their invariant requires T affinely independent and their implementation defends it, admitting a stopping point only when it is not too close to $\operatorname{aff}(T)$. Section 2 permits dependence and answers it, with a descent direction in the null space. Neither is the better choice in general: theirs avoids a case, this one resolves it.

Their linear algebra is the stronger, and it decides the regime each method is good for. They never refactorise — a dynamic QR of the edge matrix, updated by Givens rotations in $O(d^2)$ per insertion, exploiting that $\operatorname{cc}(T)$ is the orthogonal projection of c onto $\operatorname{aff}(T)$ — where Section 2 rebuilds DD^{T} and solves it afresh at every pivot. That is affordable while d is small and is why they report d up to 2 000 and this note stops at 20. In the other direction their per-pivot cost does not fall with n while the $O(nd)$ sweep vectorises, which is the regime Table 1 lives in — and where, on the evidence of its pivoting columns, the two are separated by a small constant factor rather than by anything structural, their pivot counts there running from 3 to 90. One of their observations deserves flagging: almost-cospherical clouds are their *hardest* inputs, points entering and leaving the support repeatedly, whereas cospherical inputs are where the method here is at its most accurate. Both can hold at once — theirs is a claim about iteration counts, this one about the final answer.

9 Notes

The problem is Sylvester's [7]; Elzinga and Hearn [1] gave an early finite algorithm of this type, and Sections 7 and 8 place it against the two exact lines of work that followed. Of those this method is of the quadratic-programming one, after Gärtner and Schönherr [4], whose reservation that a QP formulation needed exact arithmetic past a few hundred dimensions this note cannot dispute — its measurements stop at $d = 20$. The QP machinery is standard [6]. An implementation of the method, of the Goldfarb–Idnani core, and of the conic front end is available at `https://github.com/tschm/cvxball`.

References

[1] J. Elzinga and D. W. Hearn. Geometrical solutions for some minimax location problems. *Transportation Science* 6(4):379–394, 1972.

[2] K. Fischer, B. Gärtner and M. Kutz. Fast smallest-enclosing-ball computation in high dimensions. *Proc. ESA*, 630–641, 2003.

[3] B. Gärtner. Fast and robust smallest enclosing balls. *Proc. ESA*, 325–338, 1999.

[4] B. Gärtner and S. Schönherr. An efficient, exact, and generic quadratic programming solver for geometric optimization. *Proc. 16th Annu. ACM Sympos. Comput. Geom.*, 110–118, 2000.

[5] D. Goldfarb and A. Idnani. A numerically stable dual method for solving strictly convex quadratic programs. *Mathematical Programming* 27:1–33, 1983.

[6] J. Nocedal and S. J. Wright. *Numerical Optimization*, 2nd ed. Springer, 2006.
[7] J. J. Sylvester. A question in the geometry of situation. *Quarterly Journal of Pure and Applied Mathematics* 1:79, 1857.
[8] M. Frank and P. Wolfe. An algorithm for quadratic programming. *Naval Research Logistics Quarterly* 3:95–110, 1956.
[9] E. Welzl. Smallest enclosing disks (balls and ellipsoids). *New Results and New Trends in Computer Science*, LNCS 555, 359–370, 1991.
[10] P. Wolfe. Finding the nearest point in a polytope. *Mathematical Programming* 11:128–149, 1976.